

Frontiers in Graph Neural Networks

April 3, 2026

[Tailin Wu](#), Westlake University

Website: <https://ai4s.lab.westlake.edu.cn/course/frontiers-2026/>

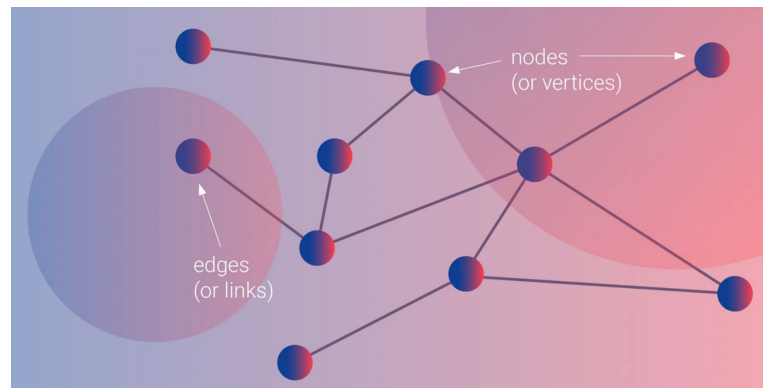


Image from: TechSlang
Some materials from [cs224w](#)

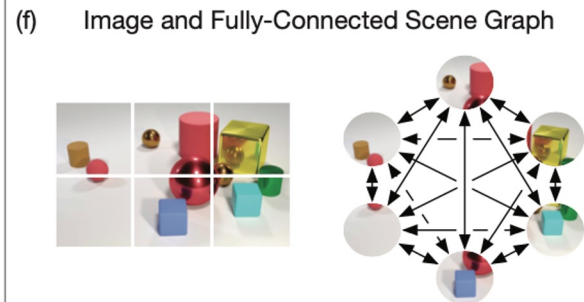
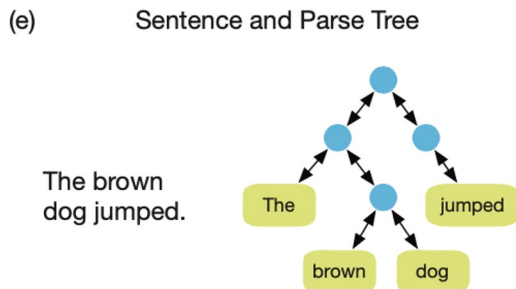
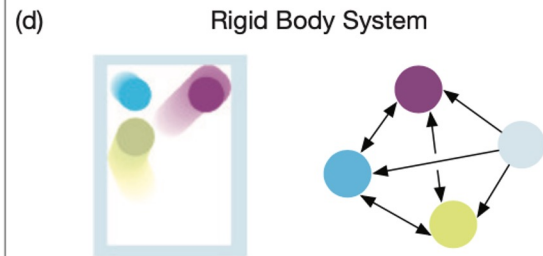
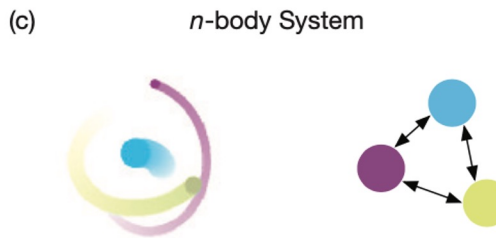
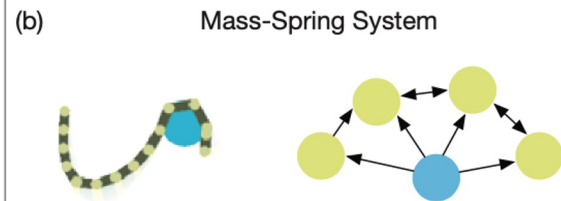
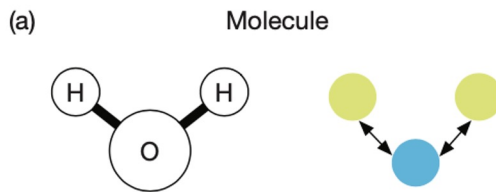
Outline

1. Introduction
2. Graph representation
3. Tasks
4. Graph Neural Networks: Fundamentals
5. Useful Techniques
6. Advanced topics
 - a. Neural subgraph matching
 - b. Graph Transformers

1. Introduction

Graph-structured systems are everywhere:

Intelligence



1. Introduction

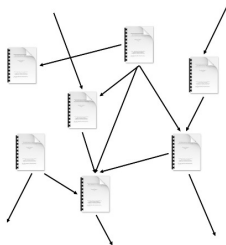
Graph-structured systems are everywhere:

Intelligence



Image credit: [Medium](#)

Social Networks



Citation Networks

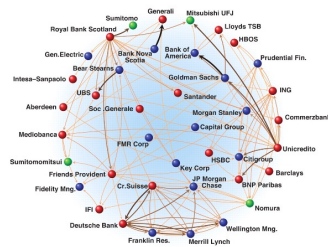


Image credit: [Science](#)

Economic Networks



Image credit: [Missoula Current News](#)

Internet



Image credit: [Lumen Learning](#)

Communication Networks

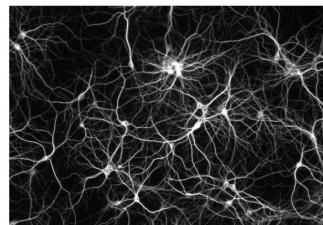


Image credit: [The Conversation](#)

Networks of Neurons

1. Introduction: Why develop graph neural networks?

Complex domains have a rich relational structure, which can be represented as a **relational graph**

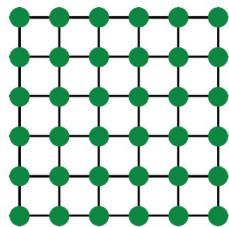
By explicitly modeling relationships we achieve better performance!

Main question:

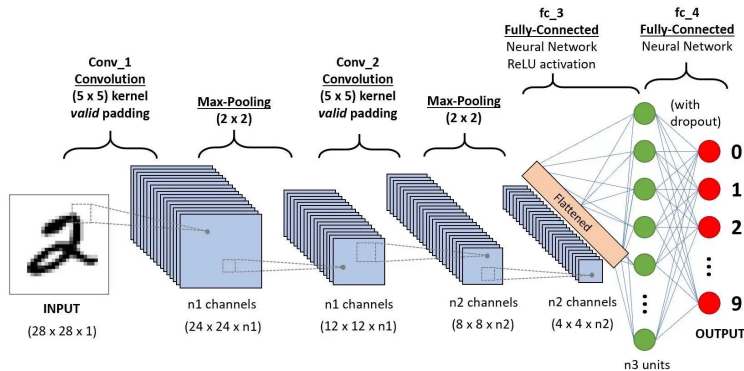
How do we take advantage of relational structure for better prediction?

1. Introduction: Why develop graph neural networks?

Modern deep learning toolbox is designed for sequences & grids:

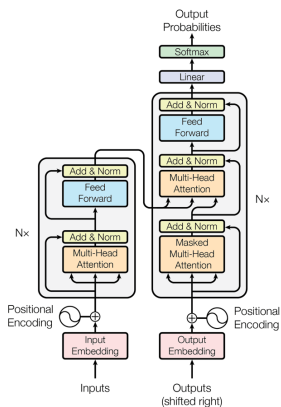


Images



CNN

Text/Speech



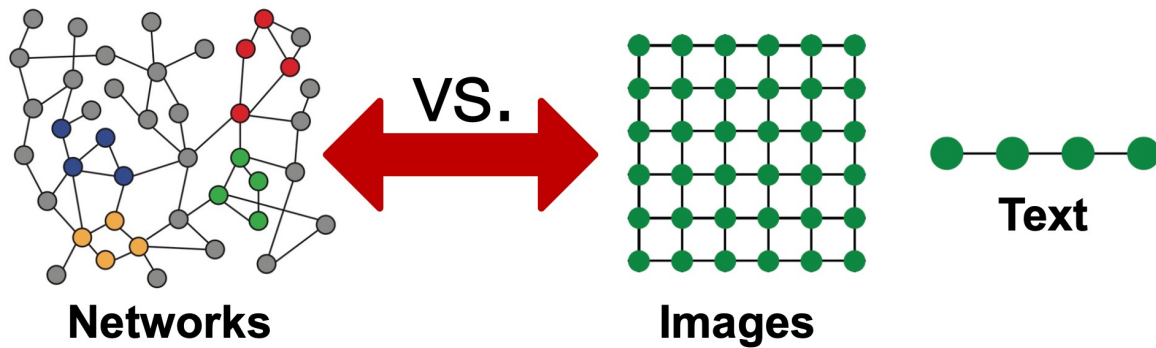
Transformer

1. Introduction: Why develop graph neural networks?

How can we develop neural networks that are much more broadly applicable and can apply to graph-structured data?

Answer: Graph Neural Networks

1. Introduction: Why is it hard?



- No fixed node ordering or reference point
- Often dynamic and have multimodal features

Outline

1. Introduction
2. Graph representation
3. Tasks
4. Graph Neural Networks: Fundamentals
5. Useful Techniques
6. Advanced topics

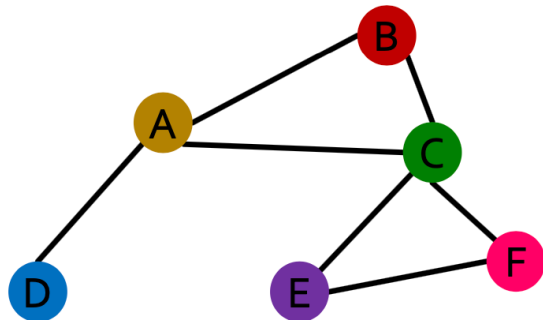
2. Graph representation

A graph $G = (V, A, X)$

- V is the **vertex set**
- A is the **adjacency matrix** (assume binary)
- $X \in \mathbb{R}^{|V| \times m}$ is a matrix of **node features**
- v : a node in V ; $N(v)$: the set of neighbors of v .
- **Node features:**
 - Social networks: User profile, User image
 - Biological networks: Gene expression profiles, gene functional information
 - When there is no node feature in the graph dataset:
 - Indicator vectors (one-hot encoding of a node)
 - Vector of constant 1: $[1, 1, \dots, 1]$

2. Graph representation

A graph $G = (V, A, X)$

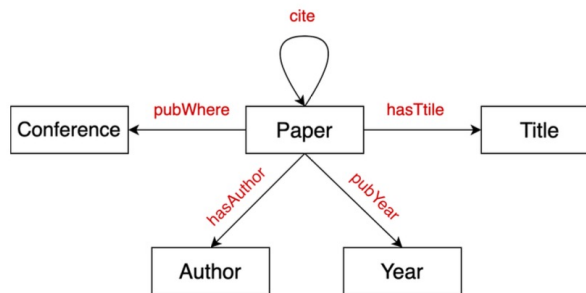
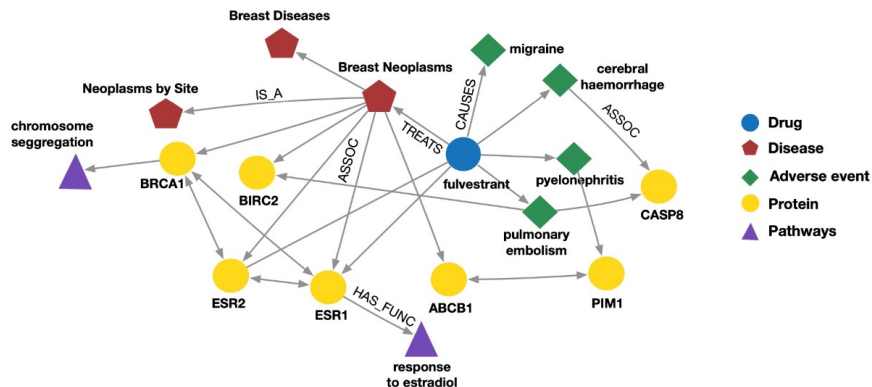


Adjacency matrix A :

$A_{ij} = 1$ if they are connected; 0 otherwise

2. Graph representation: Heterogeneous graph

A heterogeneous graph additionally has node type and edge types:



Biomedical Knowledge Graphs

Example node: **Migraine**

Example edge: **(fulvestrant, Treats, Breast Neoplasms)**

Example node type: **Protein**

Example edge type (relation): **Causes**

Academic Graphs

Example node: **ICML**

Example edge: **(GraphSAGE, NeurIPS)**

Example node type: **Author**

Example edge type (relation): **pubYear**

2. Graph representation: Heterogeneous graph

Modeling multiple objects' interaction with MeshGraphNets [1]

Example node:

vertices in the mesh

Example edge:

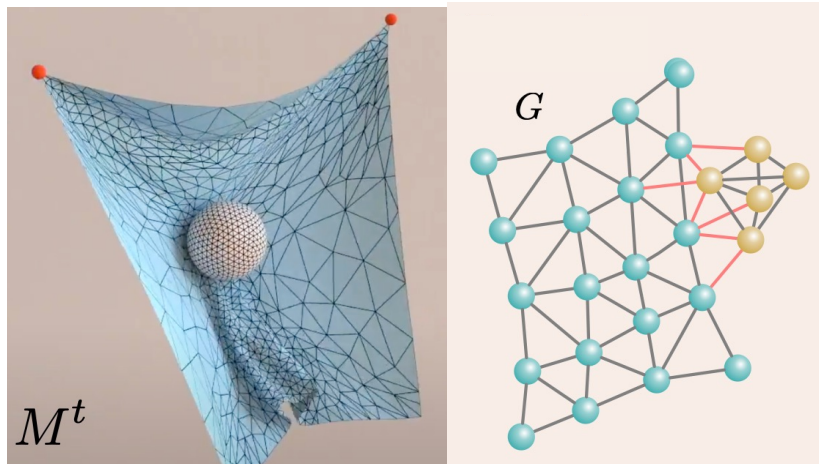
edge in and between the meshes

Example node type:

solid node

Example edge type:

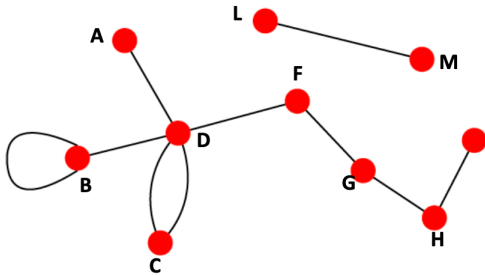
mesh-space edge, world-space edge



2. Graph representation: Directed vs. undirected graph

Undirected

- **Links:** undirected
(symmetrical, reciprocal)

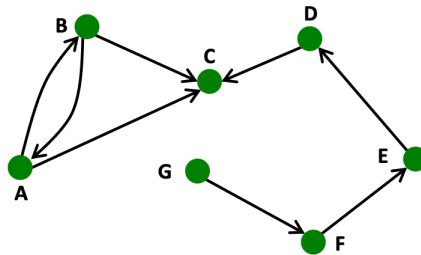


- **Other considerations:**

- Weights
- Properties

Directed

- **Links:** directed



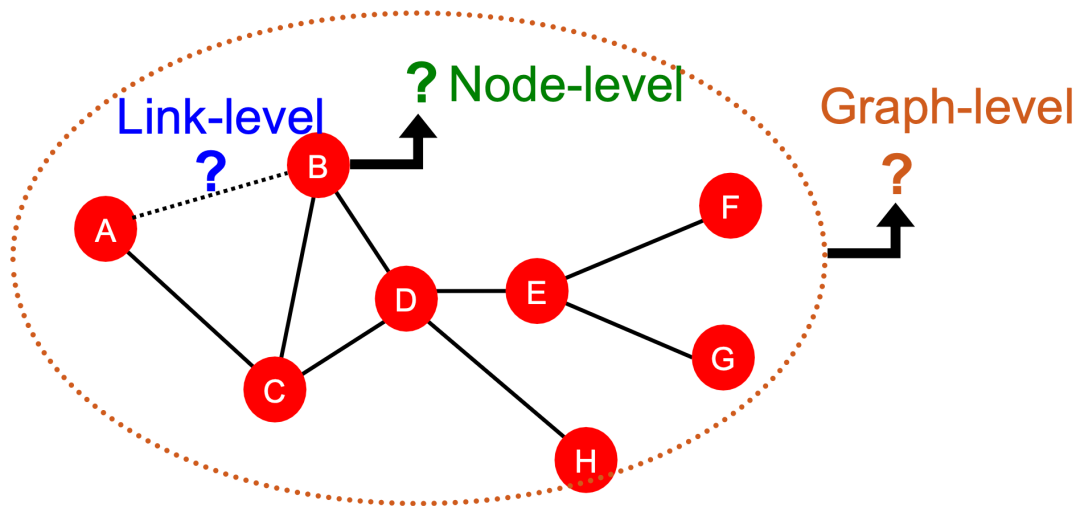
- Types
- Attributes

Outline

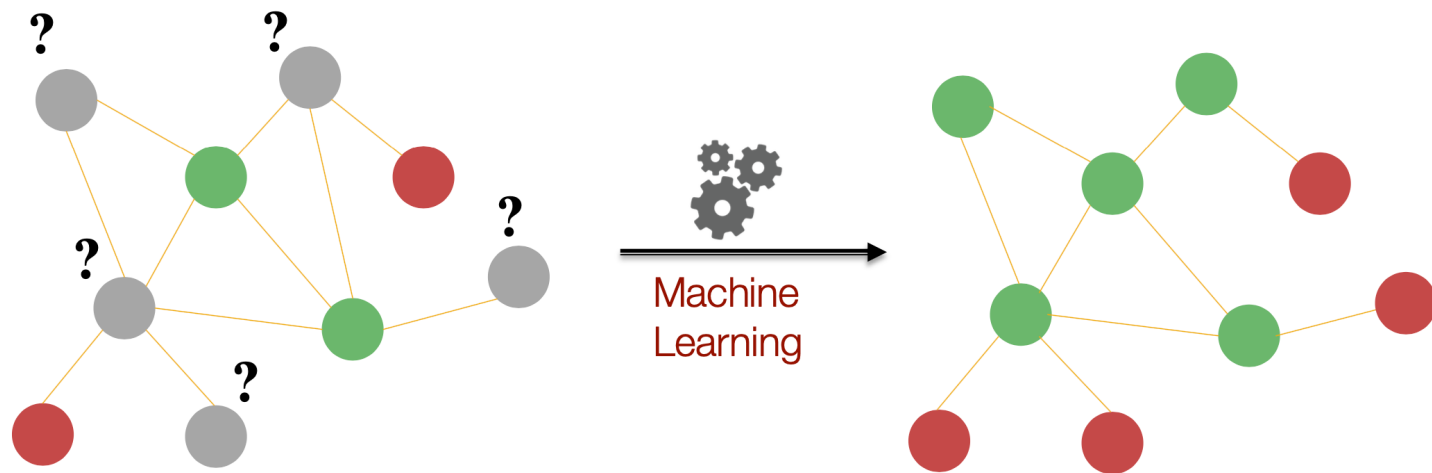
1. Introduction
2. Graph representation
3. **Tasks**
4. Graph Neural Networks: Fundamentals
5. Useful Techniques
6. Advanced topics

3. Tasks

- Node-level prediction
- Link-level prediction
- Graph-level prediction

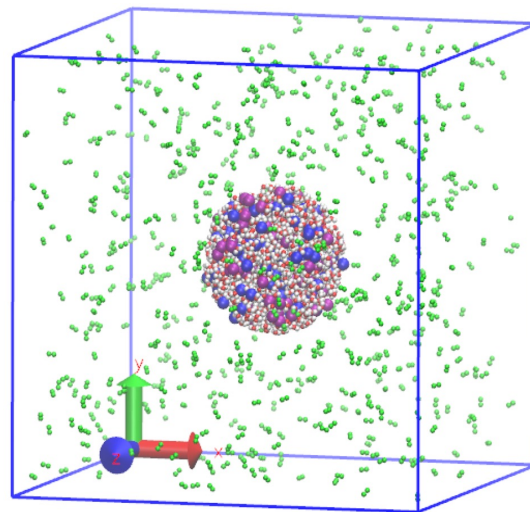
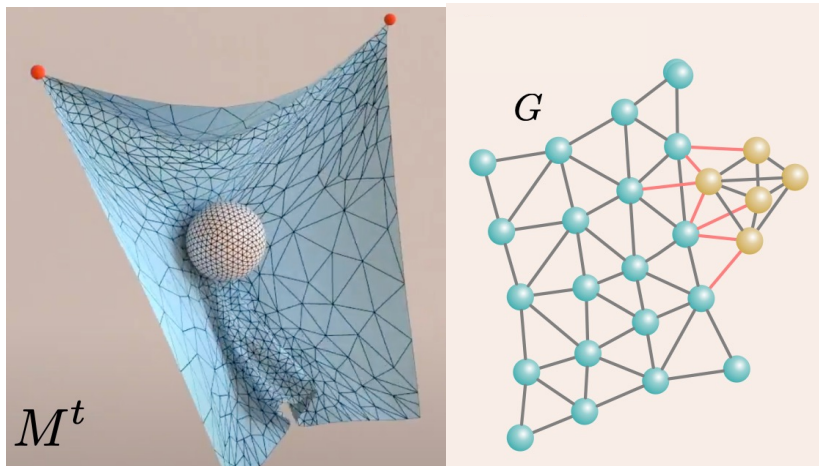


3. Task 1.1: Node classification



Node classification

3. Task 1.2: Node regression

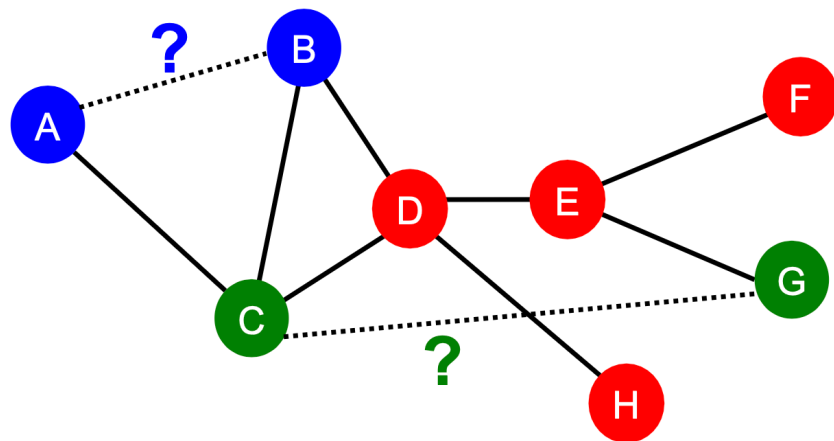


Example: Physical prediction. Predict the continuous state (position/velocity) of each node at the next time step

Application: Neural surrogate models; learning molecular dynamics [1]

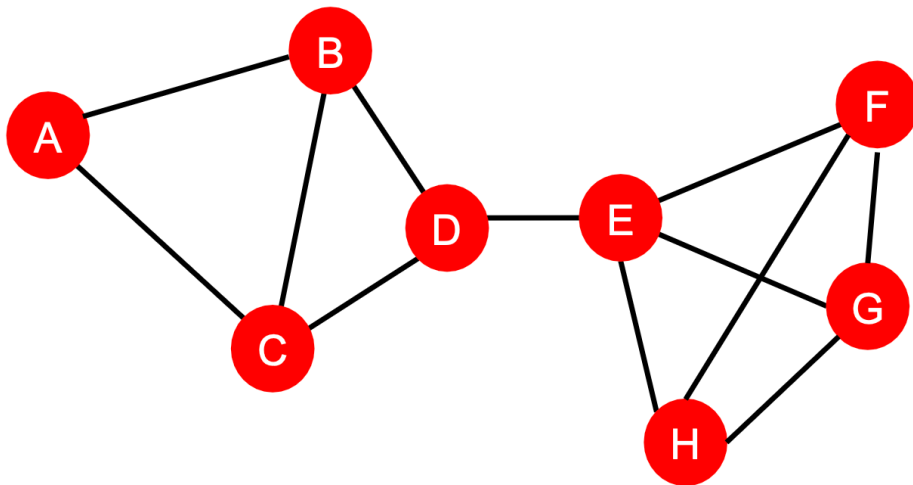
3. Task 2: Link prediction

- The task is to predict **new/missing/unknown links** based on the existing links.
- At test time, node pairs (with no existing links) are ranked, and top K node pairs are predicted.



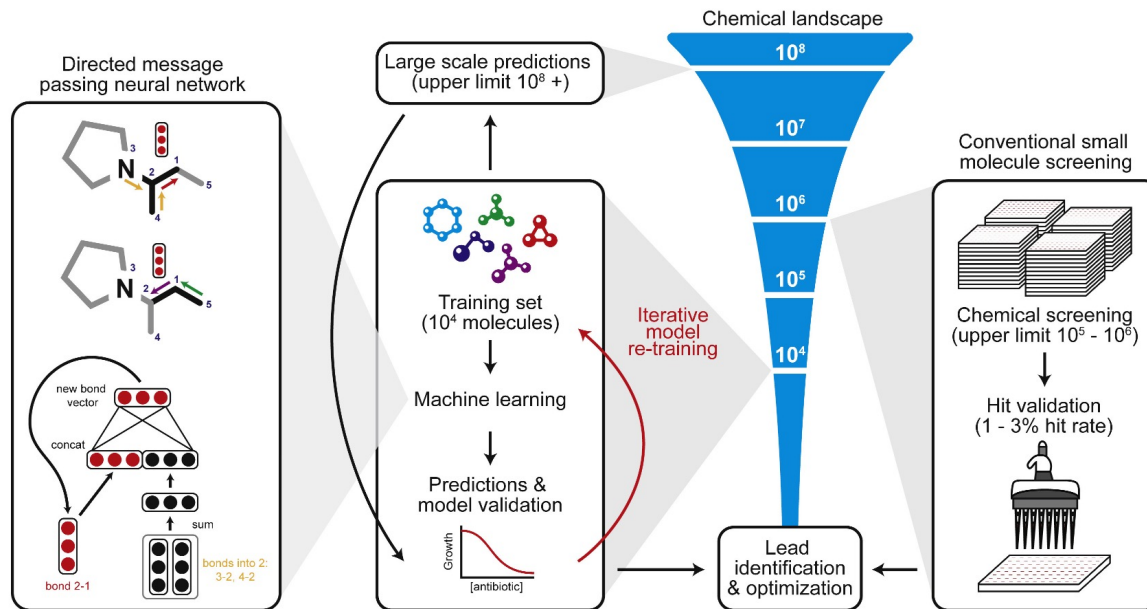
3. Task 3: Graph-level tasks

Goal: We want make a prediction for an entire graph or a subgraph of the graph.



3. Task 3: Graph-level tasks

- A Graph Neural Network **graph classification model**
- Predict promising molecules from a pool of candidates



3. Tasks: Other tasks

- Graph generation: generate molecules, proteins, etc.
- Missing node detection

Outline

1. Introduction
2. Graph representation
3. Tasks
4. Graph Neural Networks: Fundamentals
 - a. Overview
 - b. Aggregation
 - c. GCN
 - d. GraphSAGE
 - e. GAT
5. Useful Techniques
6. Advanced topics

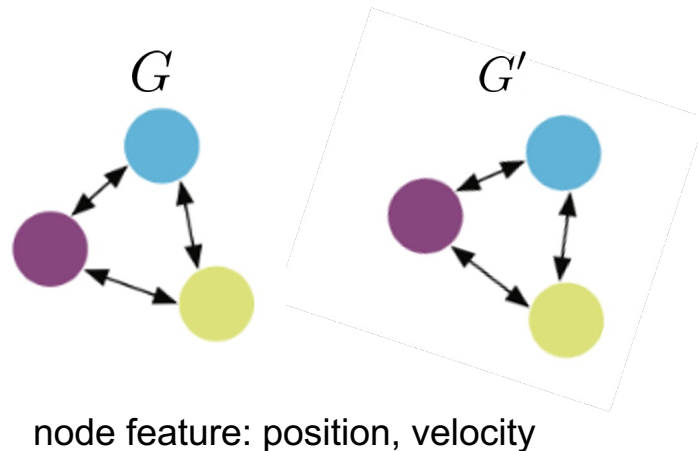
Graph Neural Networks: Overview

A GNN f_θ takes **graph-structured data** G as input, and typically maps to another G' :

$$G' = f_\theta(G)$$

n-body system:

1. Compute **forces** on each **edge**
2. **Accumulate** forces on each **node**
3. Update the **node feature** (position, velocity) based on the **accumulated force** and **previous node feature**



l^{th} message-passing layer for a generic GNN:

1. Compute learnable **messages** on each **edge**

$$m_{i \rightarrow j}^{(l)} = \text{edgeMLP}_{\theta}^{(l)} \left([n_i^{(l)}, n_j^{(l)}, e_{ij}] \right)$$

2. **Accumulate messages** on each **node**

$$z_j^{(l)} = \sum_{i \in \mathcal{N}(j)} m_{i \rightarrow j}^{(l)}$$

3. Perform a learnable update on the **node feature** using the **accumulated messages** and **previous node feature**

$$n_j^{(l+1)} = \text{nodeMLP}_{\theta}^{(l)} \left([n_i^{(l)}, z_j^{(l)}] \right)$$

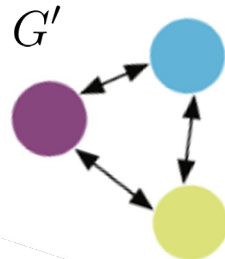
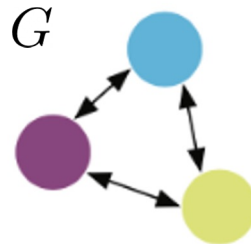
The GNN learns $\{ \text{edgeMLP}_{\theta}^{(l)}, \text{nodeMLP}_{\theta}^{(l)} \}_{l=0}^{L-1}$ by minimizing the prediction loss w.r.t. the parameter θ

$$\text{loss} = \mathbb{E}[\text{MSE}(f_{\theta}(u^t), u^{t+1})]$$

$n_i^{(l)}$: i^{th} node feature at layer l

$n_i^{(0)}$: input node feature

e_{ij} : input edge feature from node i to node j

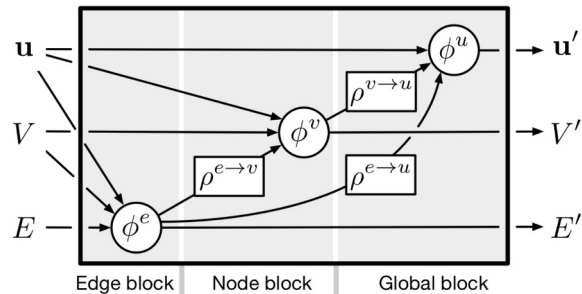
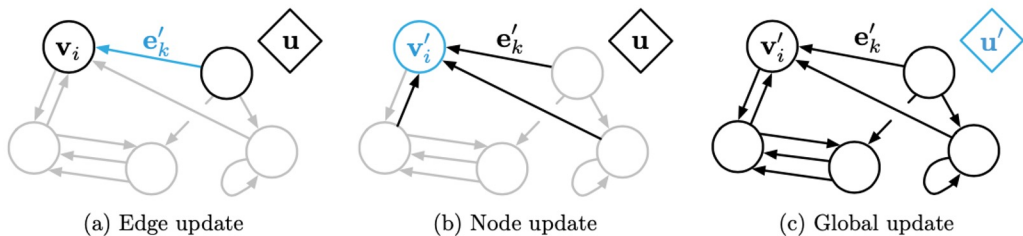


node feature: e.g., position, velocity

Battaglia *et al.* (2018)

Graph Neural Networks: Overview

Most general GNN:



$$\mathbf{x}_i^{(k)} = \gamma^{(k)} \left(\mathbf{x}_i^{(k-1)}, \square_{j \in \mathcal{N}(i)} \phi^{(k)} \left(\mathbf{x}_i^{(k-1)}, \mathbf{x}_j^{(k-1)}, \mathbf{e}_{i,j} \right) \right)$$

- Message ϕ
- Aggregate $\square$
- Update γ

Graph Neural Networks: Create it with PyG

```
import torch
from torch_geometric.nn import MessagePassing

class GNNLayer(MessagePassing):
    def __init__(self, in_channels, out_channels):
        super().__init__(aggr='add') # Choose from "Add", "Max", "Mean".

    def forward(self, x, edge_index):
        """
        out = self.propagate(edge_index, x=x)
        """
        return out

    def message(self, x_i, x_j, norm):
        """
        return message_out
```


Outline

1. Introduction
2. Graph representation
3. Tasks
4. Graph Neural Networks: Fundamentals
 - a. Overview
 - b. Aggregation
 - c. GCN
 - d. GraphSage
 - e. GAT
5. Useful Techniques
6. Advanced topics

Graph Neural Networks: Aggregation

1. **Sum**
2. **Mean**
3. **Max**
4. Median
5. LSTM
6. Sort
7. Sampling
 - a. Sample with replacement (GraphSAGE)
 - b. With dropout
8. Generalized-mean

Graph Neural Networks: Aggregation

Generalised f-Mean Aggregation:

$$\bigoplus_{i \in [1..n]} x_i = f^{-1} \left(n^{\alpha-1} \sum_{i \in [1..n]} f(x_i - \beta \mu) \right)$$

- α parameter controls its level of dependence on the cardinality of the input X
- The β parameter enables AFM to calculate *centralised moments*, which are quantitative measures of the distribution of the input X

[1] Generalised f-Mean Aggregation for Graph Neural Networks, NeurIPS 2023

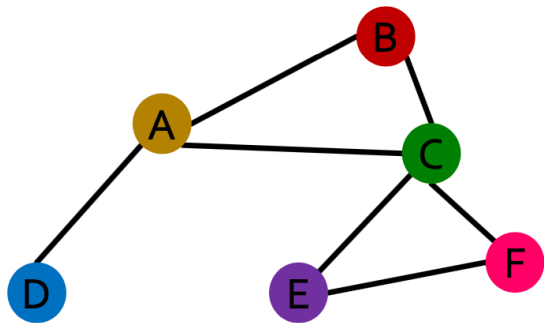
Aggregation Function	α	β	f
mean: $\frac{1}{n} \sum x_i$	0	0	$f(x) = x$
sum: $\sum x_i$	1	0	$f(x) = x$
product: $\prod x_i $	1	0	$f(x) = \log(x)$
min (magnitude): $\min x_i $	0	0	$f(x) = \lim_{p \rightarrow \infty} x ^{-p}$
max (magnitude): $\max x_i $	0	0	$f(x) = \lim_{p \rightarrow \infty} x ^p$
min: $\min x_i$	0	0	$f(x) = \lim_{p \rightarrow \infty} e^{-px}$
max: $\max x_i$	0	0	$f(x) = \lim_{p \rightarrow \infty} e^{px}$
harmonic mean: $\frac{n}{\sum \frac{1}{x_i}}$	0	0	$f(x) = \frac{1}{x}$
geometric mean: $\sqrt[n]{\prod x_i }$	0	0	$f(x) = \log(x)$
root mean square: $\sqrt{\frac{1}{n} \sum x_i^2}$	0	0	$f(x) = x^2$
euclidean norm: $\sqrt{\sum x_i^2}$	1	0	$f(x) = x^2$
standard deviation: $\sqrt{\frac{1}{n} \sum (x_i - \mu)^2}$	0	1	$f(x) = x^2$
log-sum-exp: $\log \left(\sum e^{x_i} \right)$	1	0	$f(x) = e^x$

Outline

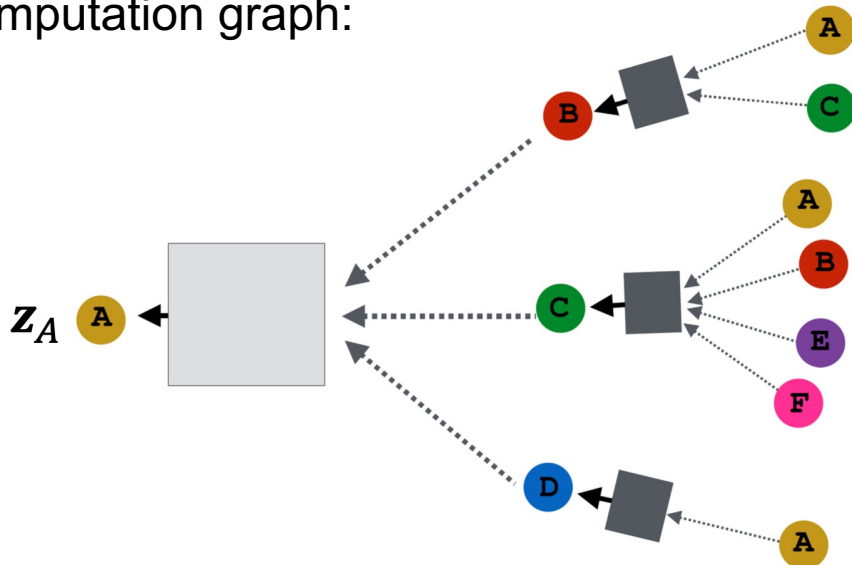
1. Introduction
2. Graph representation
3. Tasks
4. Graph Neural Networks: Fundamentals
 - a. Overview
 - b. Aggregation
 - c. GCN
 - d. GraphSAGE
 - e. GAT
5. Graph Neural Networks: Advanced Techniques

Graph Neural Networks: Computation graph

Input:



Computation graph:

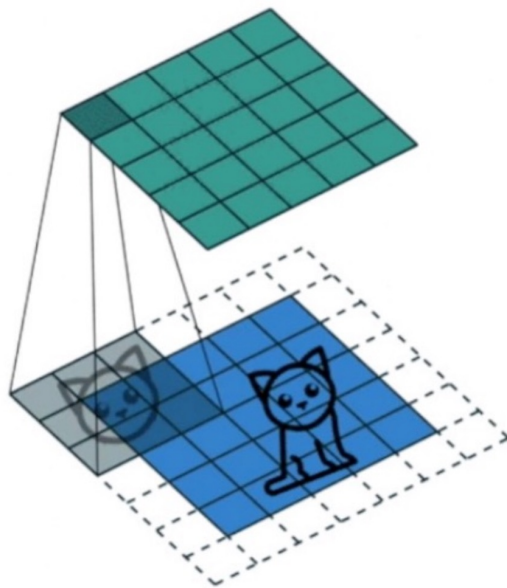


Inspiration

Inspiration: Convolutional Neural Network (CNN, lecture 7) and adapt for graphs

A convolution layer:

- with a sliding window
- abstract templates
- similarity measured by dot product
- stronger activation, better matching



Graph Convolutional Networks (GCN)

Trainable weight matrices
(i.e., what we learn)

$$h_v^{(0)} = x_v$$
$$h_v^{(k+1)} = \sigma\left(W_k \sum_{u \in N(v)} \frac{h_u^{(k)}}{|N(v)|} + B_k h_v^{(k)}\right), \forall k \in \{0..K-1\}$$

Final node embedding

W_k : weight matrix for neighborhood aggregation

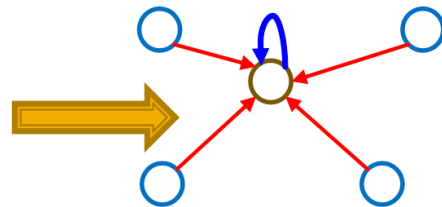
B_k : weight matrix for transforming hidden vector of self

Graph Convolutional Networks (GCN)

Rewriting in Matrix form:

$$H^{(k+1)} = \sigma(\tilde{A}H^{(k)}W_k^T + H^{(k)}B_k^T)$$

where $\tilde{A} = D^{-1}A$



$$H^{(k)} = [h_1^{(k)} \dots h_{|V|}^{(k)}]^T$$

A : adjacency matrix, $A_{ij} = 1$ iff connected

GCN in PyG

```
class GCNConv(MessagePassing):
    def __init__(self, in_channels, out_channels):
        super().__init__(aggr='add') # "Add" aggregation (Step 5).
        self.lin = Linear(in_channels, out_channels, bias=False)
        self.bias = Parameter(torch.empty(out_channels))

        self.reset_parameters()

    def forward(self, x, edge_index):
        # x has shape [N, in_channels]
        # edge_index has shape [2, E]

        # Step 1: Add self-loops to the adjacency matrix.
        edge_index, _ = add_self_loops(edge_index, num_nodes=x.size(0))

        # Step 2: Linearly transform node feature matrix.
        x = self.lin(x)

        # Step 3: Compute normalization.
        row, col = edge_index
        deg = degree(col, x.size(0), dtype=x.dtype)
        deg_inv_sqrt = deg.pow(-0.5)
        deg_inv_sqrt[deg_inv_sqrt == float('inf')] = 0
        norm = deg_inv_sqrt[row] * deg_inv_sqrt[col]

        # Step 4-5: Start propagating messages.
        out = self.propagate(edge_index, x=x, norm=norm)

        # Step 6: Apply a final bias vector.
        out = out + self.bias

        return out

    def message(self, x_j, norm):
        # x_j has shape [E, out_channels]

        # Step 4: Normalize node features.
        return norm.view(-1, 1) * x_j
```

How to train GNN, supervised

Node embedding $\mathbf{z}_v$ is a function of input graph

Supervised setting: We want to minimize loss $\mathcal{L}$:

$$\min_{\Theta} \mathcal{L}(\mathbf{y}, f_{\Theta}(\mathbf{z}_v))$$

- $\mathbf{y}$: node label
- $\mathcal{L}$ could be L2 if $\mathbf{y}$ is real number, or cross-entropy if $\mathbf{y}$ is categorical

How to train GNN, unsupervised

Unsupervised setting:

- No node label available
- **Use the graph structure as the supervision!**

$$\min_{\Theta} \mathcal{L} = \sum_{z_u, z_v} \text{CE}(y_{u,v}, \text{DEC}(z_u, z_v))$$

- where $y_{u,v} = 1$ when node u and v are **similar**
- $z_u = f_{\Theta}(u)$ and $\text{DEC}(\cdot, \cdot)$ is the dot product

CE is the cross entropy loss:

- $\text{CE}(\mathbf{y}, f(\mathbf{x})) = -\sum_{i=1}^C (y_i \log f_{\Theta}(x)_i)$
 - y_i and $f_{\Theta}(x)_i$ are the **actual** and **predicted** values of the i -th class.
 - **Intuition:** the lower the loss, the closer the prediction is to one-hot

Outline

1. Introduction
2. Graph representation
3. Tasks
4. Graph Neural Networks: Fundamentals
 - a. Overview
 - b. Aggregation
 - c. GCN
 - d. GraphSAGE
 - e. GAT
5. Useful Techniques
6. Advanced topics

GraphSAGE

$$\mathbf{h}_v^{(l)} = \sigma \left(\mathbf{W}^{(l)} \cdot \text{CONCAT} \left(\mathbf{h}_v^{(l-1)}, \text{AGG} \left(\left\{ \mathbf{h}_u^{(l-1)}, \forall u \in N(v) \right\} \right) \right) \right)$$

- **How to write this as Message + Aggregation?**

- **Message** is computed within the $\text{AGG}(\cdot)$

- **Two-stage aggregation**

- **Stage 1:** Aggregate from node neighbors

$$\mathbf{h}_{N(v)}^{(l)} \leftarrow \text{AGG} \left(\left\{ \mathbf{h}_u^{(l-1)}, \forall u \in N(v) \right\} \right)$$

- **Stage 2:** Further aggregate over the node itself

$$\mathbf{h}_v^{(l)} \leftarrow \sigma \left(\mathbf{W}^{(l)} \cdot \text{CONCAT}(\mathbf{h}_v^{(l-1)}, \mathbf{h}_{N(v)}^{(l)}) \right)$$

GraphSAGE : Aggregation

- **Mean:** Take a weighted average of neighbors

$$\text{AGG} = \sum_{u \in N(v)} \frac{\mathbf{h}_u^{(l-1)}}{|N(v)|}$$

Aggregation Message computation

- **Pool:** Transform neighbor vectors and apply symmetric vector function Mean($\cdot$) or Max($\cdot$)

$$\text{AGG} = \text{Mean}(\{\text{MLP}(\mathbf{h}_u^{(l-1)}), \forall u \in N(v)\})$$

Aggregation Message computation

- **LSTM:** Apply LSTM to reshuffled of neighbors

$$\text{AGG} = \text{LSTM}([\mathbf{h}_u^{(l-1)}, \forall u \in \pi(N(v))])$$

Aggregation

GraphSAGE: L2 normalization

■ ℓ_2 Normalization:

- **Optional:** Apply ℓ_2 normalization to $\mathbf{h}_v^{(l)}$ at every layer
- $\mathbf{h}_v^{(l)} \leftarrow \frac{\mathbf{h}_v^{(l)}}{\|\mathbf{h}_v^{(l)}\|_2} \quad \forall v \in V$ where $\|u\|_2 = \sqrt{\sum_i u_i^2}$ (ℓ_2 -norm)
- Without ℓ_2 normalization, the embedding vectors have different scales (ℓ_2 -norm) for vectors
- In some cases (not always), normalization of embedding results in performance improvement
- After ℓ_2 normalization, all vectors will have the same ℓ_2 -norm

Outline

1. Introduction
2. Graph representation
3. Tasks
4. Graph Neural Networks: Fundamentals
 - a. Overview
 - b. Aggregation
 - c. GCN
 - d. GraphSAGE
 - e. GAT
5. Useful Techniques
6. Advanced topics

Limitation of prior method

$$\mathbf{h}_v^{(l)} = \sigma(\sum_{u \in N(v)} \alpha_{vu} \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)})$$

Attention weights

■ In GCN / GraphSAGE

- $\alpha_{vu} = \frac{1}{|N(v)|}$ is the **weighting factor (importance)** of node u 's message to node v
- $\Rightarrow \alpha_{vu}$ is defined **explicitly** based on the structural properties of the graph (node degree)
- $\Rightarrow$ All neighbors $u \in N(v)$ are equally important to node v

Limitation of prior method

$$\mathbf{h}_v^{(l)} = \sigma(\sum_{u \in N(v)} \alpha_{vu} \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)})$$

Attention weights

Not all node's neighbors are equally important

- **Attention** is inspired by cognitive attention.
- The **attention** α_{vu} focuses on the important parts of the input data and fades out the rest.
 - **Idea:** the NN should devote more computing power on that small but important part of the data.
 - Which part of the data is more important depends on the context and is learned through training.

Graph Attention Network (GAT): Goal

Can we do better than simple neighborhood aggregation?

Can weighting factors α_{vu} be learned?

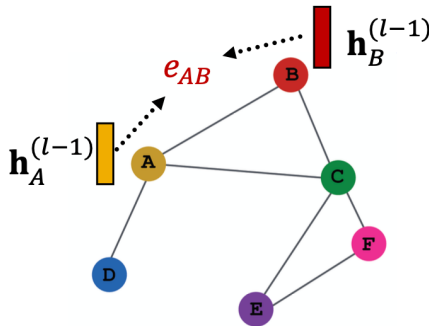
- **Goal:** Specify **arbitrary importance** to different neighbors of each node in the graph
- **Idea:** Compute embedding $\mathbf{h}_v^{(l)}$ of each node in the graph following an **attention strategy**:
 - Nodes attend over their neighborhoods' message
 - Implicitly specifying different weights to different nodes in a neighborhood

GAT: Attention mechanism (1)

- Let α_{vu} be computed as a byproduct of an **attention mechanism a** :
 - (1) Let a compute **attention coefficients e_{vu}** across pairs of nodes u, v based on their messages:

$$e_{vu} = a(\mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)}, \mathbf{W}^{(l)} \mathbf{h}_v^{(l-1)})$$

- e_{vu} indicates the importance of u 's message to node v



$$e_{AB} = a(\mathbf{W}^{(l)} \mathbf{h}_A^{(l-1)}, \mathbf{W}^{(l)} \mathbf{h}_B^{(l-1)})$$

GAT: Attention mechanism (2)

- **Normalize** e_{vu} into the **final attention weight** α_{vu}

- Use the **softmax** function, so that $\sum_{u \in N(v)} \alpha_{vu} = 1$:

$$\alpha_{vu} = \frac{\exp(e_{vu})}{\sum_{k \in N(v)} \exp(e_{vk})}$$

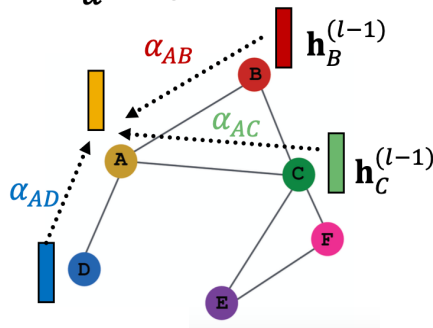
- **Weighted sum** based on the **final attention weight**

α_{vu} :

$$\mathbf{h}_v^{(l)} = \sigma(\sum_{u \in N(v)} \alpha_{vu} \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)})$$

Weighted sum using α_{AB} , α_{AC} , α_{AD} :

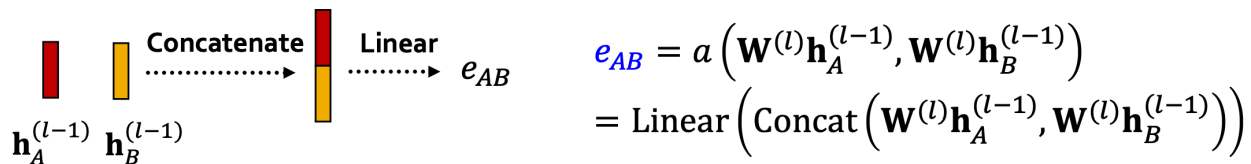
$$\mathbf{h}_A^{(l)} = \sigma(\alpha_{AB} \mathbf{W}^{(l)} \mathbf{h}_B^{(l-1)} + \alpha_{AC} \mathbf{W}^{(l)} \mathbf{h}_C^{(l-1)} + \alpha_{AD} \mathbf{W}^{(l)} \mathbf{h}_D^{(l-1)})$$



GAT: Attention mechanism (3)

- **What is the form of attention mechanism a ?**

- The approach is agnostic to the choice of a
 - E.g., use a simple single-layer neural network
 - a have trainable parameters (weights in the Linear layer)



- **Parameters of a are trained jointly:**
 - Learn the parameters together with weight matrices (i.e., other parameter of the neural net $\mathbf{W}^{(l)}$) in an end-to-end fashion

GAT: Attention mechanism (4)

- **Multi-head attention:** Stabilizes the learning process of attention mechanism

- **Create multiple attention scores** (each replica with a different set of parameters):

$$\mathbf{h}_v^{(l)}[1] = \sigma(\sum_{u \in N(v)} \alpha_{vu}^1 \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)})$$

$$\mathbf{h}_v^{(l)}[2] = \sigma(\sum_{u \in N(v)} \alpha_{vu}^2 \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)})$$

$$\mathbf{h}_v^{(l)}[3] = \sigma(\sum_{u \in N(v)} \alpha_{vu}^3 \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)})$$

- **Outputs are aggregated:**
 - By concatenation or summation
 - $\mathbf{h}_v^{(l)} = \text{AGG}(\mathbf{h}_v^{(l)}[1], \mathbf{h}_v^{(l)}[2], \mathbf{h}_v^{(l)}[3])$

GAT: Benefits of Attention

- **Key benefit:** Allows for (implicitly) specifying **different importance values (α_{vu}) to different neighbors**
- **Computationally efficient:**
 - Computation of attentional coefficients can be parallelized across all edges of the graph
 - Aggregation may be parallelized across all nodes
- **Storage efficient:**
 - Sparse matrix operations do not require more than $O(V + E)$ entries to be stored
 - **Fixed** number of parameters, irrespective of graph size
- **Localized:**
 - Only **attends over local network neighborhoods**
- **Inductive capability:**
 - It is a shared *edge-wise* mechanism
 - It does not depend on the global graph structure

Outline

1. Introduction
2. Graph representation
3. Tasks
4. Graph Neural Networks: Fundamentals
5. Useful Techniques
 - a. Dropout
 - b. Skip connections
 - c. Virtual node
 - d. Neighborhood sampling
6. Advanced topics

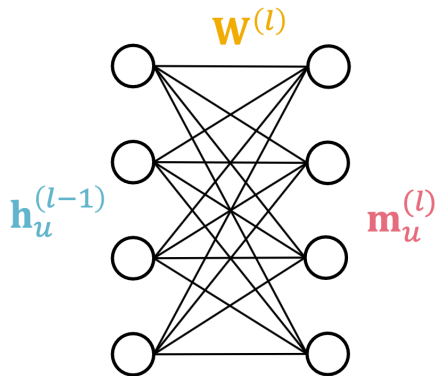
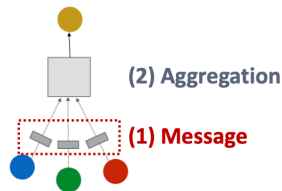
5. Useful techniques: Dropout

a. Dropout

- In GNN, Dropout is applied to **the linear layer in the message function**

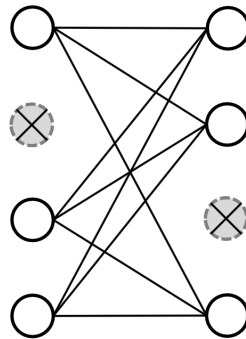
- A simple message function with linear

layer: $\mathbf{m}_u^{(l)} = \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)}$



Visualization of a linear layer

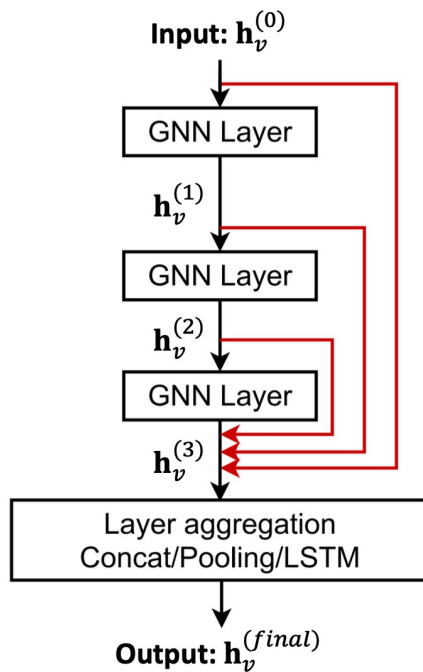
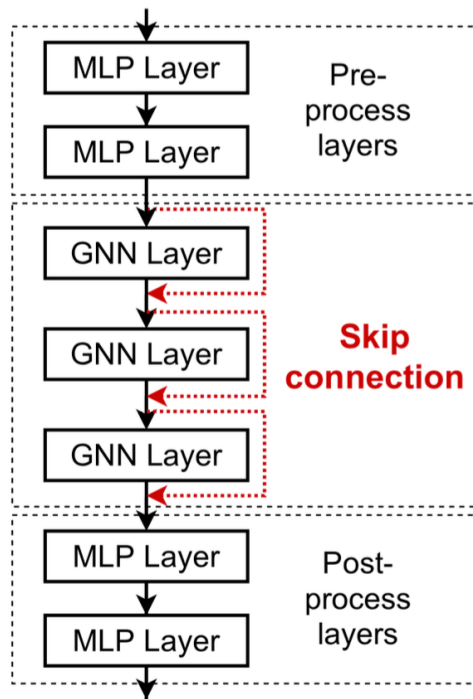
Dropout



5. Useful techniques: Skip connections

b. Adding skip connections

Benefits: Node embeddings in earlier GNN layers can sometimes better differentiate nodes



Two ways of adding skip connections

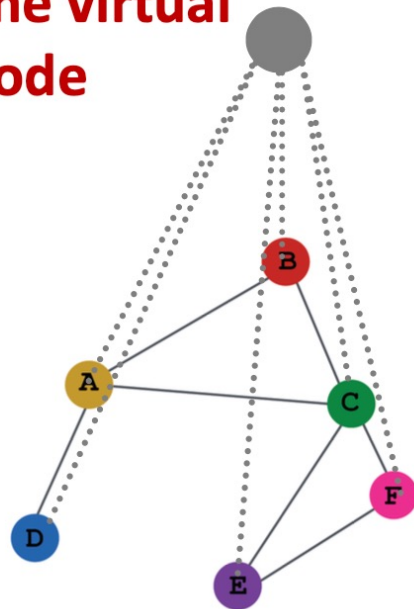
5. Useful techniques: Virtual node

c. Virtual node

The virtual node will connect to all the nodes in the graph

Benefits: Greatly **improves message passing in sparse graphs**

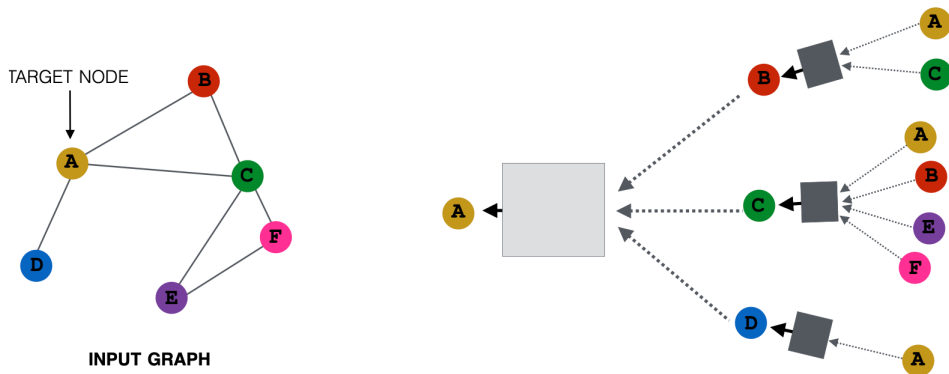
The virtual node



5. Useful techniques: Neighborhood sampling

d. Neighborhood sampling

- **Our approach so far:**
 - All the neighbors are used for message passing
- **Problem: Dense/large graphs, high-degree nodes**

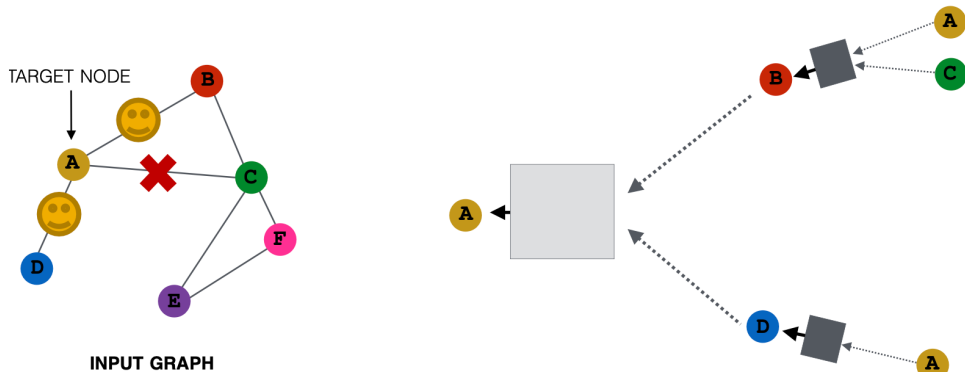


- **New idea:** (Randomly) determine a node's neighborhood for message passing

5. Useful techniques: Neighborhood sampling

d. Neighborhood sampling

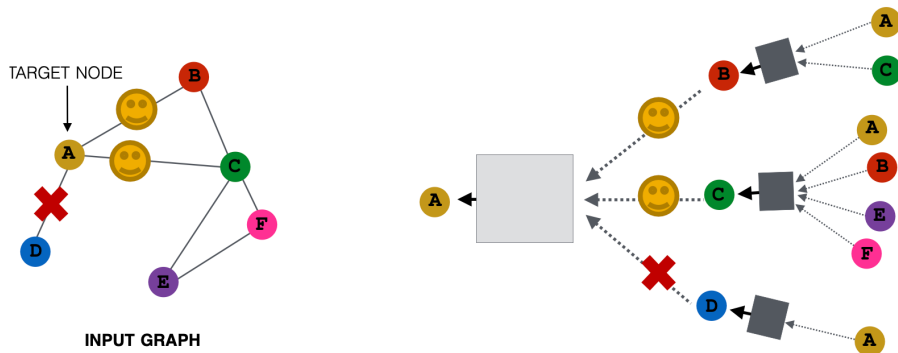
- **For example, we can randomly choose 2 neighbors to pass messages**
- **Only nodes B and D will pass message to A**



5. Useful techniques: Neighborhood sampling

d. Neighborhood sampling

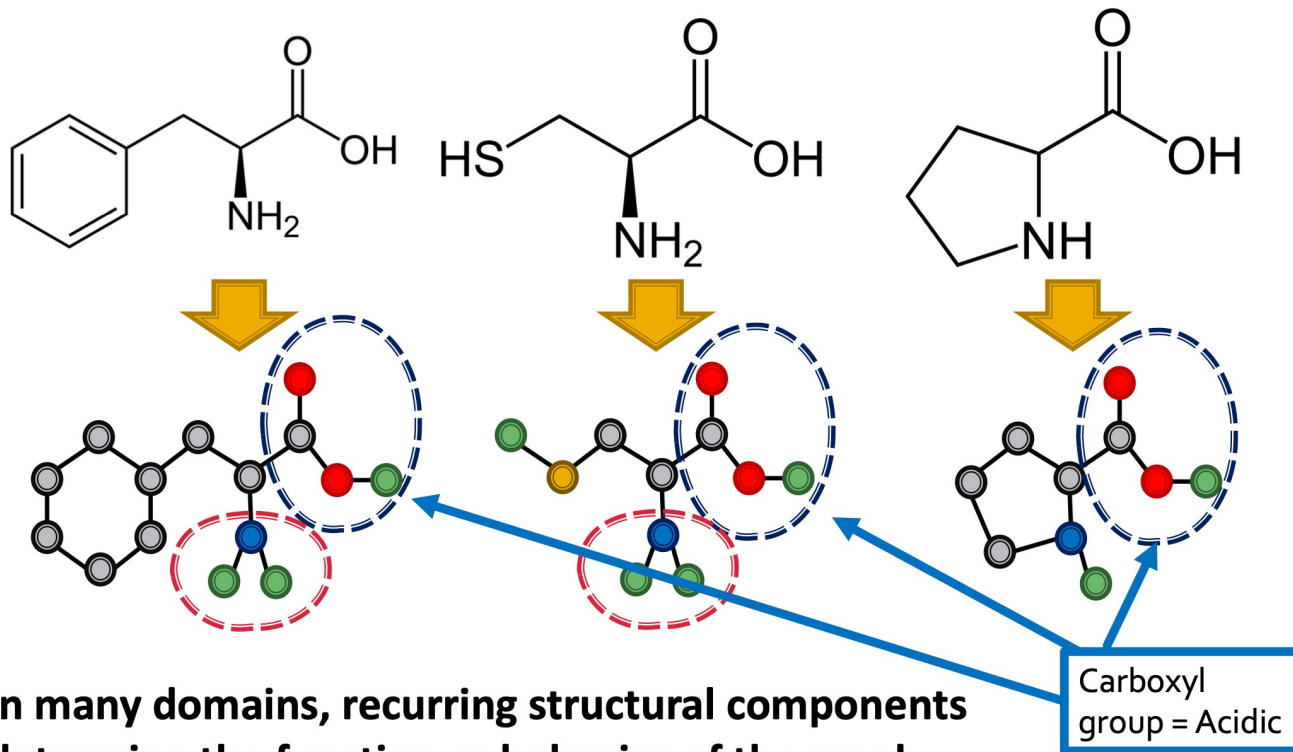
- In expectation, we can get embeddings similar to the case where all the neighbors are used
 - **Benefits:** Greatly reduce computational cost
 - And in practice it works great!



Outline

1. Introduction
2. Graph representation
3. Tasks
4. Graph Neural Networks: Fundamentals
5. Useful Techniques
6. Advanced topics
 - a. Neural subgraph matching
 - b. Graph Transformers

Subgraph: Building blocks of networks



In many domains, recurring structural components determine the function or behavior of the graph

Subgraph definition: Node-induced subgraph

Two ways to formalize "network building blocks"

- Given graph $G = (V, E)$:

Def 1. Node-induced subgraph: Take subset of the **nodes** and all edges induced by the nodes:

- $G' = (V', E')$ is a node induced subgraph iff
 - $V' \subseteq V$
 - $E' = \{(u, v) \in E \mid u, v \in V'\}$
 - G' is the subgraph of G **induced** by V'
- **Alternate terminology:** "induced subgraph"

Subgraph definition: Edge-induced subgraph

Def 2. Edge-induced subgraph: Take subset of the edges and all corresponding nodes

- $G' = (V', E')$ is an edge induced subgraph iff
 - $E' \subseteq E$
 - $V' = \{v \in V \mid (v, u) \in E' \text{ for some } u\}$
- **Alternate terminology:** "non-induced subgraph" or just "subgraph"

Subgraph definition

Two ways to formalize "network building blocks"

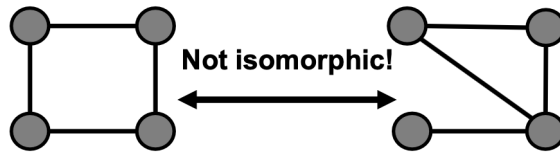
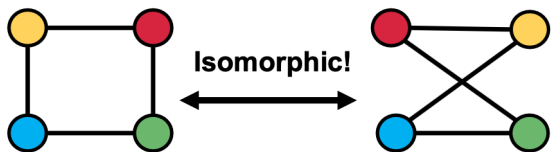
- The best definition depends on the domain!

Examples:

- **Chemistry**: Node-induced (functional groups)
- **Knowledge graphs**: Often edge-induced (focus is on edges representing logical relations)

Graph isomorphism

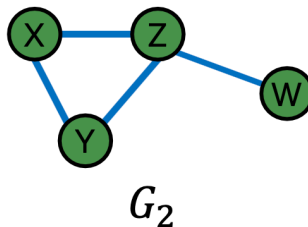
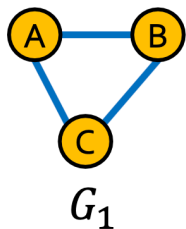
- **Graph isomorphism problem:** Check whether two graphs are identical:
 - $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are **isomorphic** if there exists a **bijection** $f: V_1 \rightarrow V_2$ such that $(u, v) \in E_1$ iff $(f(u), f(v)) \in E_2$
 - f is called the **isomorphism**:



- We do not know if graph isomorphism is NP-hard, nor is any polynomial algorithm found for solving graph isomorphism.

Subgraph isomorphism

- G_2 is **subgraph-isomorphic** to G_1 if some subgraph of G_2 is isomorphic to G_1
 - We also commonly say G_1 is a subgraph of G_2
 - We can use either the node-induced or edge-induced definition of subgraph
 - **This problem is NP-hard**



$f:$

V_1	V_2
A	X
B	Y
C	Z

A-B-C matches with X-Y-Z: There is a subgraph isomorphism between G_1 and G_2 .

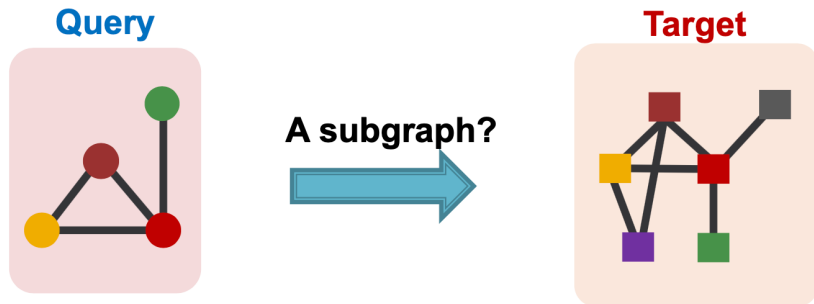
Subgraph matching: Task setting

Given:

- Large **target** graph (can be disconnected)
- **Query** graph (connected)

Decide:

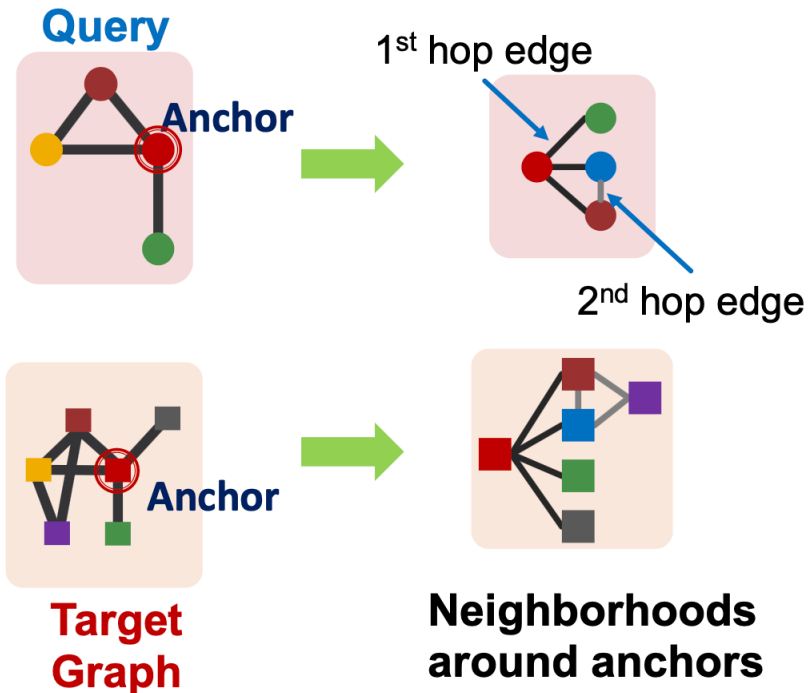
- Is a **query** graph a subgraph in the **target** graph?



- Node colors indicate the correct mapping of the nodes

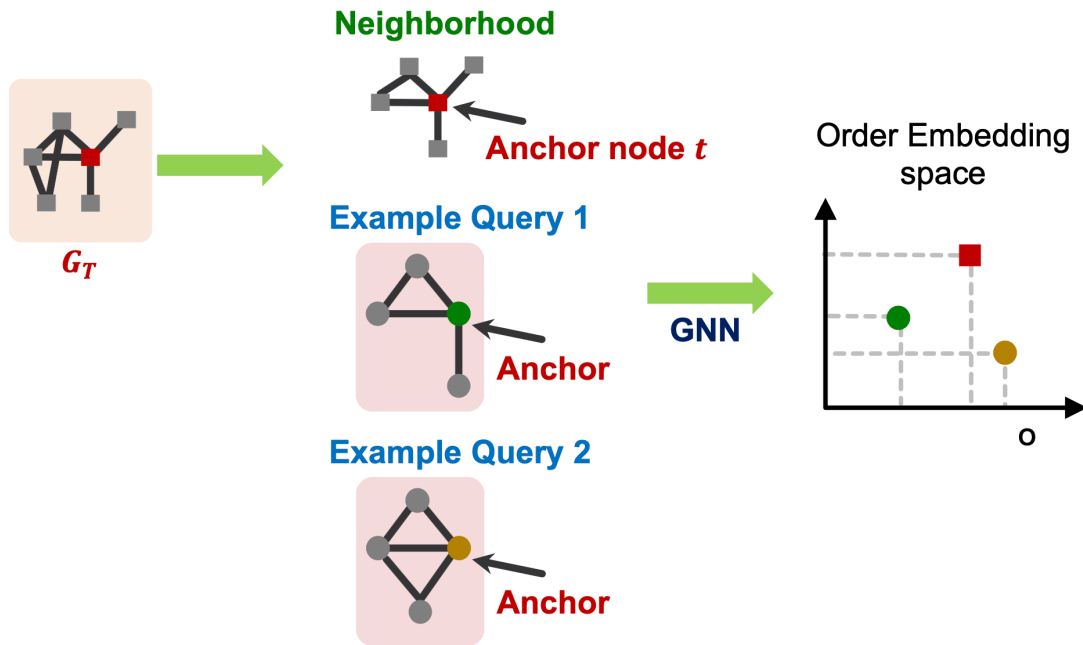
Neural subgraph matching: Node-anchored neighborhood

We utilize node-anchored neighborhood:



Neural subgraph matching: Order embedding space

Embed the node-anchored neighborhood to an **order embedding space**:



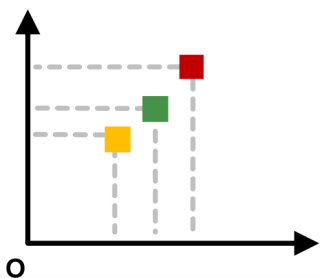
By comparing the embedding, we find that $\bullet \preceq \blacksquare$ but $\bullet \not\preceq \bullet$,
Indicating that only query 1 is a subgraph of the neighborhood of t

Why order embedding space?

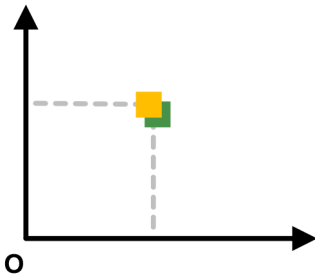
- **Subgraph isomorphism relationship** can be nicely encoded in order embedding space
 - **Transitivity**: If $\blacksquare \preceq \blacksquare$, $\blacksquare \preceq \blacksquare$ then $\blacksquare \preceq \blacksquare$
 - **Anti-symmetry**: If $\blacksquare \preceq \blacksquare$ and $\blacksquare \preceq \blacksquare$, then $\blacksquare = \blacksquare$
 - **Closure under intersection**: The 0 embedding satisfies $0 \preceq \blacksquare$ for any order embedding $\blacksquare$ since all dimensions of order embedding are non-negative
 - Corollary: If $\blacksquare \preceq \blacksquare$ and $\blacksquare \preceq \blacksquare$ then $\blacksquare$ has a valid embedding

0 embedding:
Trivial graph
with one node

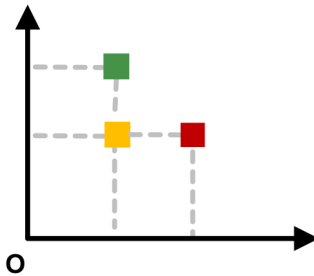
Transitivity



Anti-symmetry



Closure under intersection

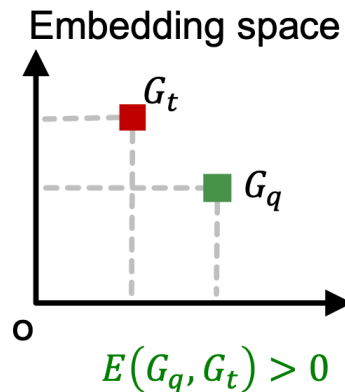
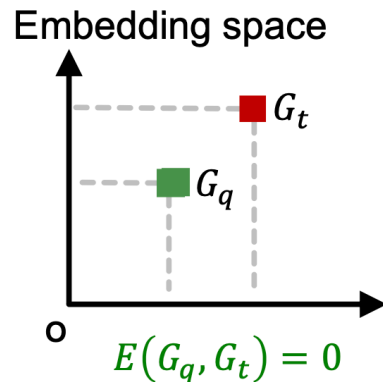


Neural subgraph matching: Training

$$E(G_q, G_t) = \sum_{i=1}^D (\max(0, z_q[i] - z_t[i]))^2$$

Train on these examples by minimizing the following **max-margin loss**:

- **For positive examples:** Minimize $E(G_q, G_t)$ when G_q is a subgraph of G_t
- **For negative examples:** Minimize $\max(0, \alpha - E(G_q, G_t))$

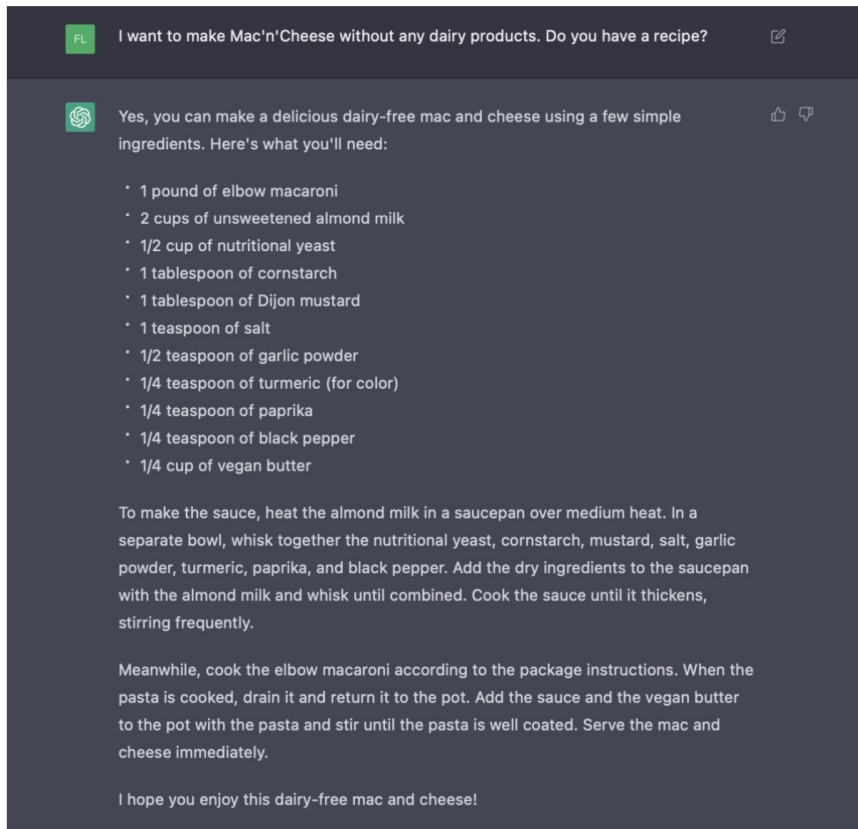


Outline

1. Introduction
2. Graph representation
3. Tasks
4. Graph Neural Networks: Fundamentals
5. Useful Techniques
6. Advanced topics
 - a. Neural subgraph matching
 - b. Graph Transformers

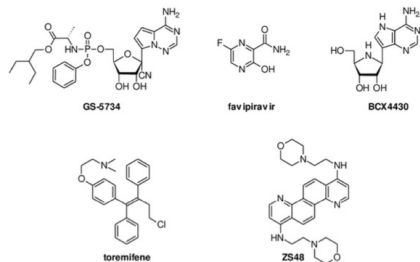
Transformers underlie today's generative AI

ChatGPT

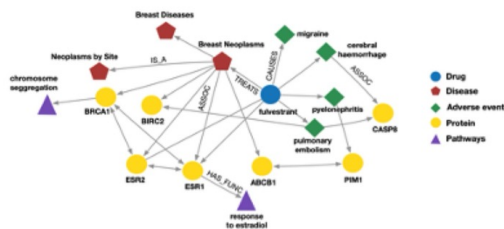


Goal: Bring power of Transformers to graphs

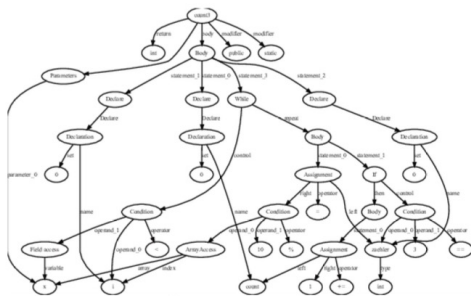
There is lots of multi-billion node/graph scale data to learn from.



Small molecules



Knowledge graphs



Code graphs

Image credit: ResearchGate



Communication networks

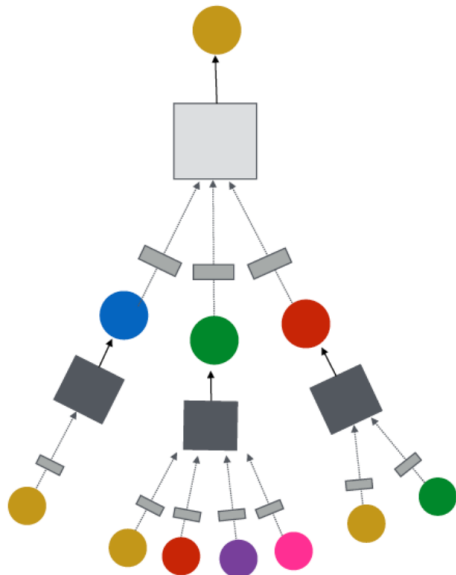
Image credit: Lumen Learning

Key elements of Transformers

- Tokenizing
- Self-attention
- Position encoding

Message passing vs. Self-attention

Message Passing



GNN

Vs.

Self-attention

$$\begin{aligned} X & \times W^Q = Q \\ X & \times W^K = K \\ X & \times W^V = V \\ \text{softmax} \left(\frac{Q \times K^T}{\sqrt{d_k}} \right) & \times V = Z \end{aligned}$$

Transformer

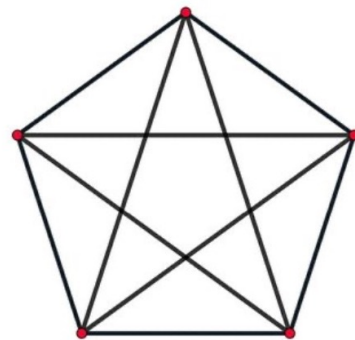
Self-attention as message passing

Self-attention can be written as message + aggregation on a fully connected graph

Consider a 5-token sequence

Each of the token's embedding attends to all other tokens

⇒ The graph is **fully connected**!

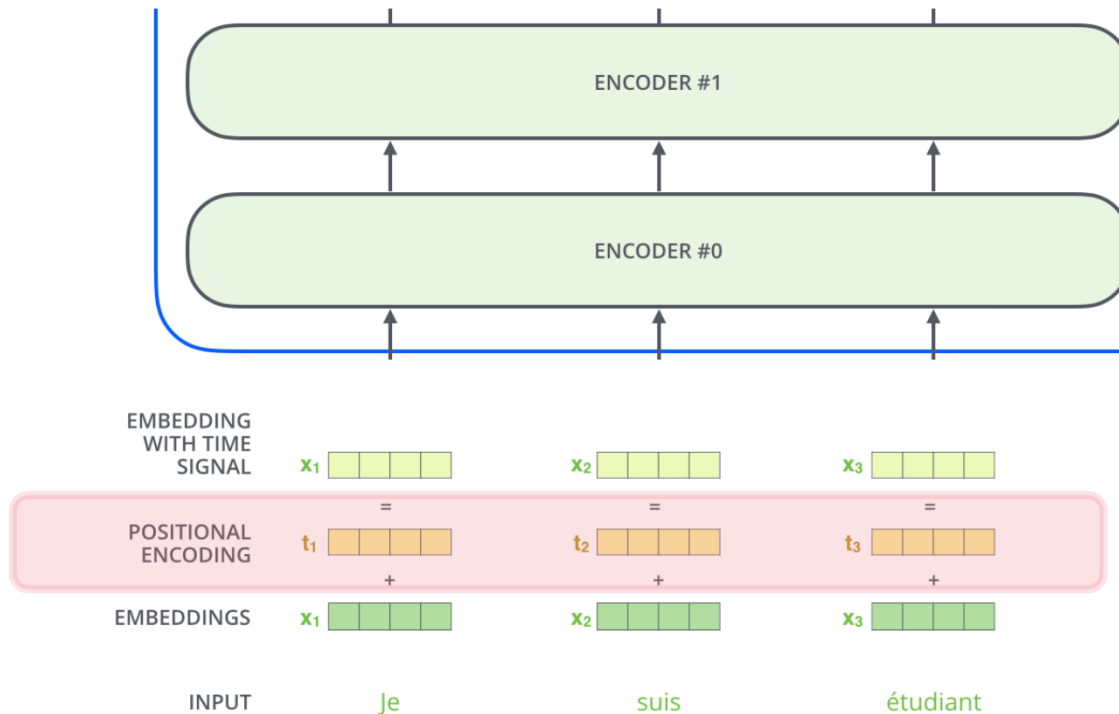


Steps for computing new embedding for token 1:

- **1. Compute message from j:** $MSG(x_j) = (v_j, k_j) = (W^V x_j, W^K x_j)$
- **2. Compute query for 1:** $q_1 = W^Q x_1$
- **3. Aggregate all messages:** $Agg(\{MSG(x_j): j\}, q_1) = \sum_{j=1}^n softmax_j(q_1^T k_j) v_j$

Position encoding for Transformers

Position encoding gives ordering for the input



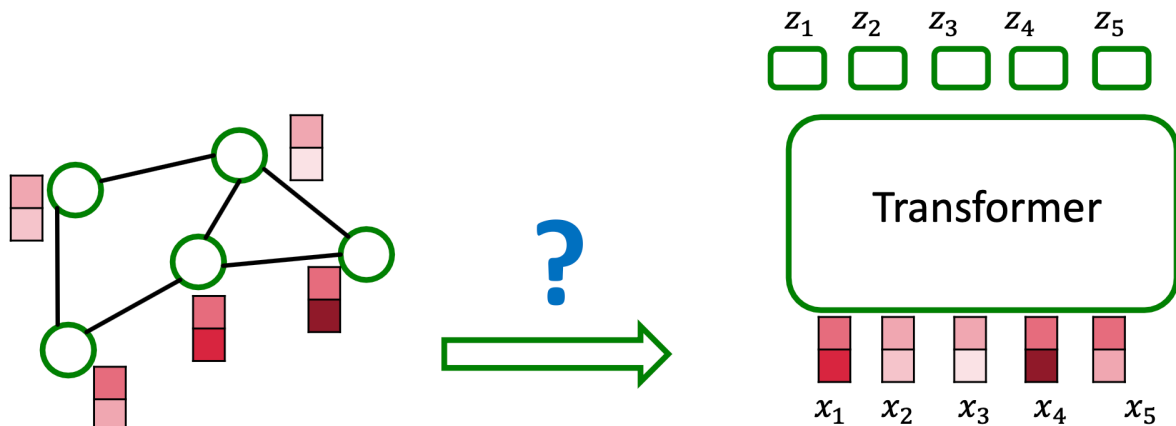
Graph Transformer

A graph Transformer must take the following inputs:

■ Key components of Transformer

- (1) Node features?  ■ (1) tokenizing
- (2) Adjacency information?  ■ (2) positional encoding
- (3) Edge features?  ■ (3) self-attention

Graph Transformer: Node features as tokens



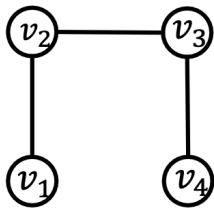
(1) Input tokens = Node features

Graph Transformer: Adjacency as position encoding

Utilize Laplacian eigenvector position encodings

Key object: Laplacian Matrix $L = \text{Degrees} - \text{Adjacency}$

- Each graph has its own Laplacian matrix
- Laplacian encodes the graph structure
- Several Laplacian variants that add degree information differently



$L =$

1	0	0	0
0	2	0	0
0	0	2	0
0	0	0	1

Degree of each node

$-$

0	1	0	0
1	0	1	0
0	1	0	1
0	0	1	0

Adjacency

Graph Transformer: Laplacian eigenvector

- Laplacian matrix captures graph structure
- **Its eigenvectors inherit this structure**
- This is important because eigenvectors are vectors (!) and so can be fed into a Transformer
- Eigenvectors with small eigenvalue = local structure, large eigenvalue = global symmetries

Refresher

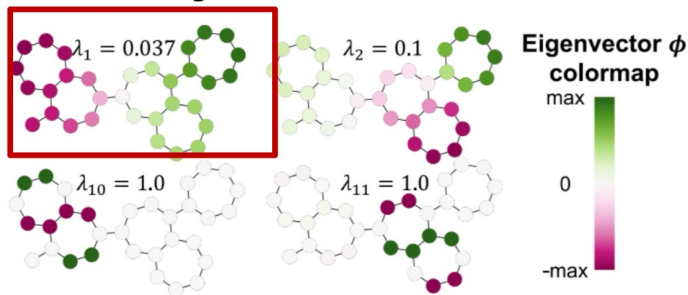
Eigenvector: v such that $Lv = \lambda v$

$L: n \times n$ matrix

$v: n$ dimensional vector

λ : Scalar eigenvalue

Visualize one eigenvector

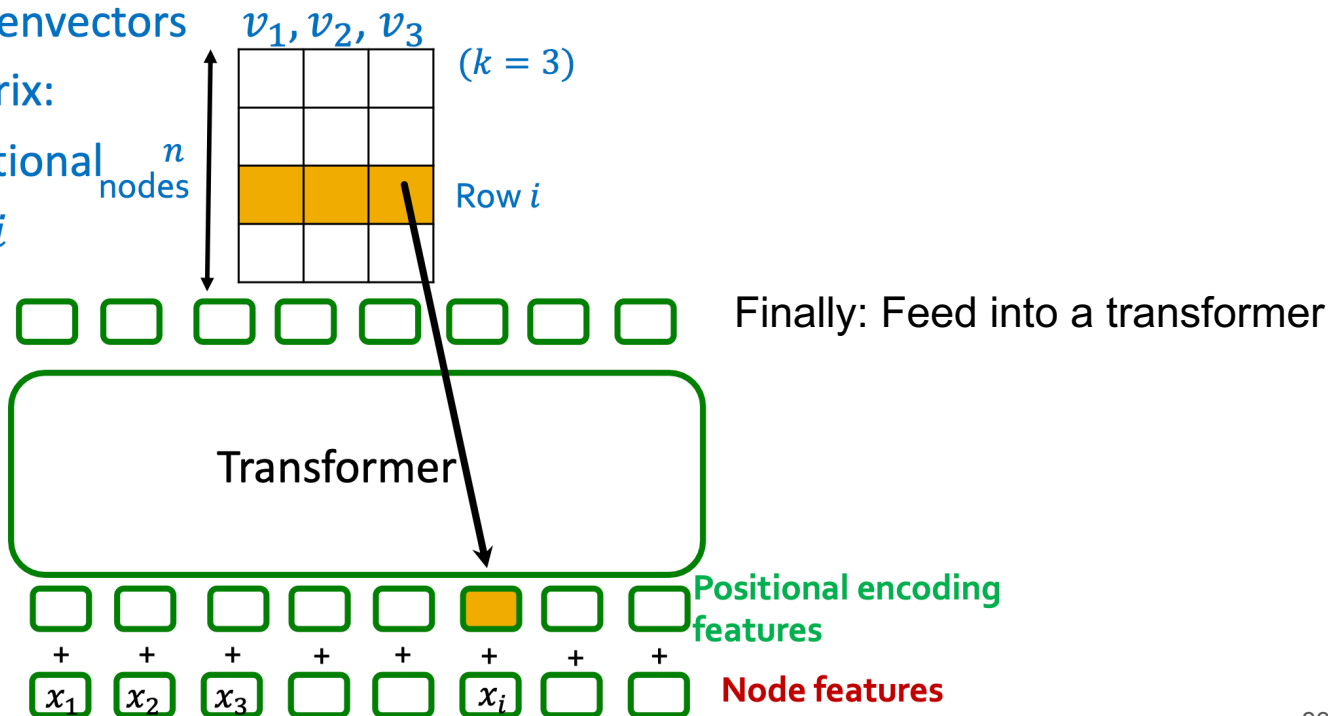
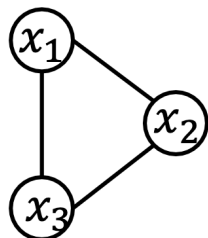


(Figure from Kreuzer* and Beaini* et al. 2021)

Graph Transformer

Positional encoding steps:

- 1. compute k eigenvectors
- 2. Stack into matrix:
- 3. i th row is positional encoding for node i



Graph Transformer: Important literature

Important literature:

- Kreuzer, Devin, et al. "Rethinking graph transformers with spectral attention." NeurIPS 2021
- Lim, Derek, et al. "Sign and basis invariant networks for spectral graph representation learning." ICLR 2023 spotlight
- Ma, Liheng, et al. "Graph inductive biases in transformers without message passing." International Conference on Machine Learning. PMLR, 2023.
- Bo, Deyu, et al. "Specformer: Spectral graph neural networks meet transformers." ICLR 2023
- Chen, Jinsong, et al. "NAGphormer: A tokenized graph transformer for node classification in large graphs." ICLR 2023

Graph Transformer: Important literature

Important literature:

- Kreuzer, Devin, et al. "Rethinking graph transformers with spectral attention." NeurIPS 2021
- Lim, Derek, et al. "Sign and basis invariant networks for spectral graph representation learning." ICLR 2023 spotlight
- Ma, Liheng, et al. "Graph inductive biases in transformers without message passing." International Conference on Machine Learning. PMLR, 2023.
- Bo, Deyu, et al. "Specformer: Spectral graph neural networks meet transformers." ICLR 2023
- Chen, Jinsong, et al. "NAGphormer: A tokenized graph transformer for node classification in large graphs." ICLR 2023

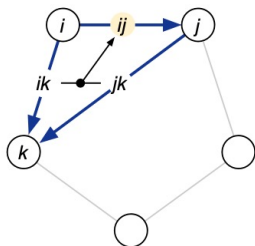
Outline

1. Introduction
2. Graph representation
3. Tasks
4. Graph Neural Networks: Fundamentals
5. Useful Techniques
6. Advanced topics
 - a. Neural subgraph matching
 - b. Graph Transformers

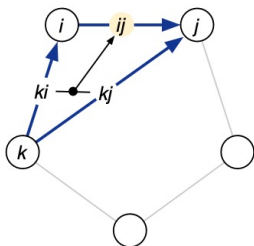
Other advanced topics: Hypergraphs

- Have hyper-edges (edges that involve more than two nodes)
- Perform message passing from higher-order edge to lower-order edges.
- For example, in AlphaFold 2 [1], the relations between edges are also important.
- Therefore, first perform message passing from hyperedge (order 2) to edge (order 1), then perform message passing from edge to nodes (order 0).

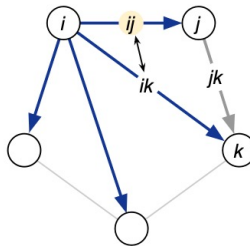
Triangle multiplicative update using 'outgoing' edges



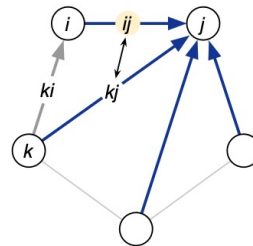
Triangle multiplicative update using 'incoming' edges



Triangle self-attention around starting node

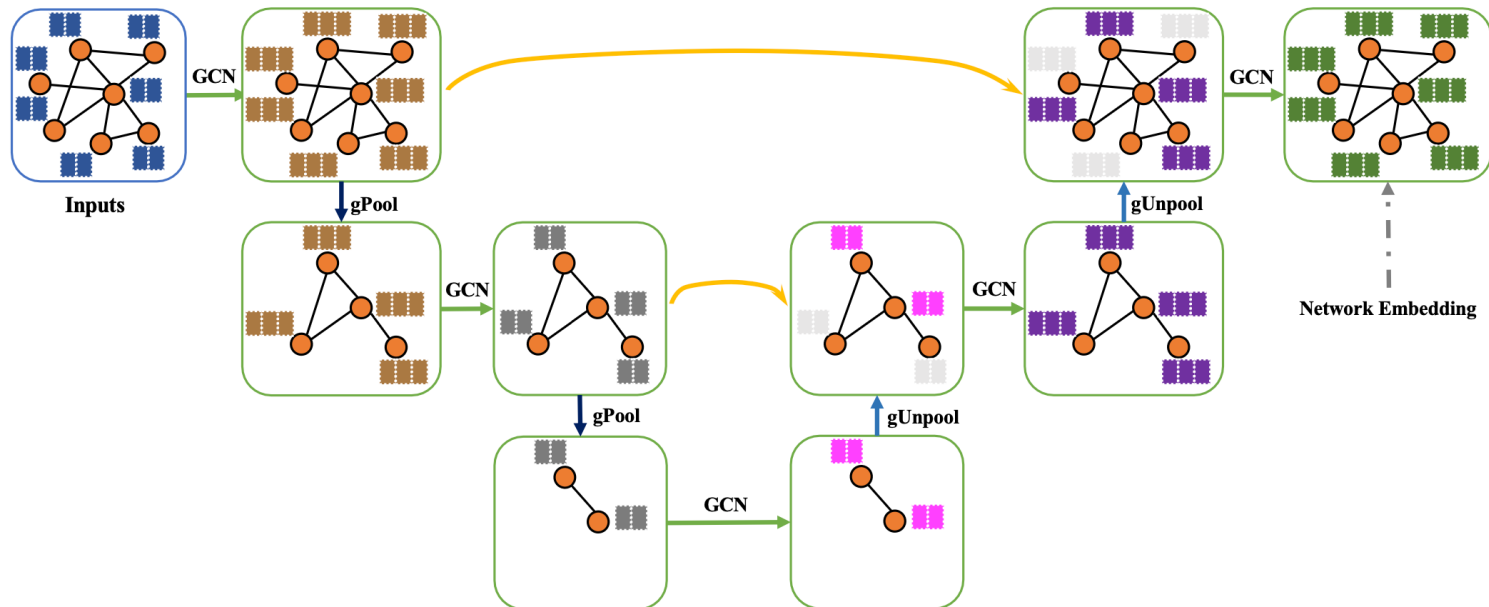


Triangle self-attention around ending node



Other advanced topics: Hierarchical pooling

Similar to U-Net, we have Graph U-Net [1]

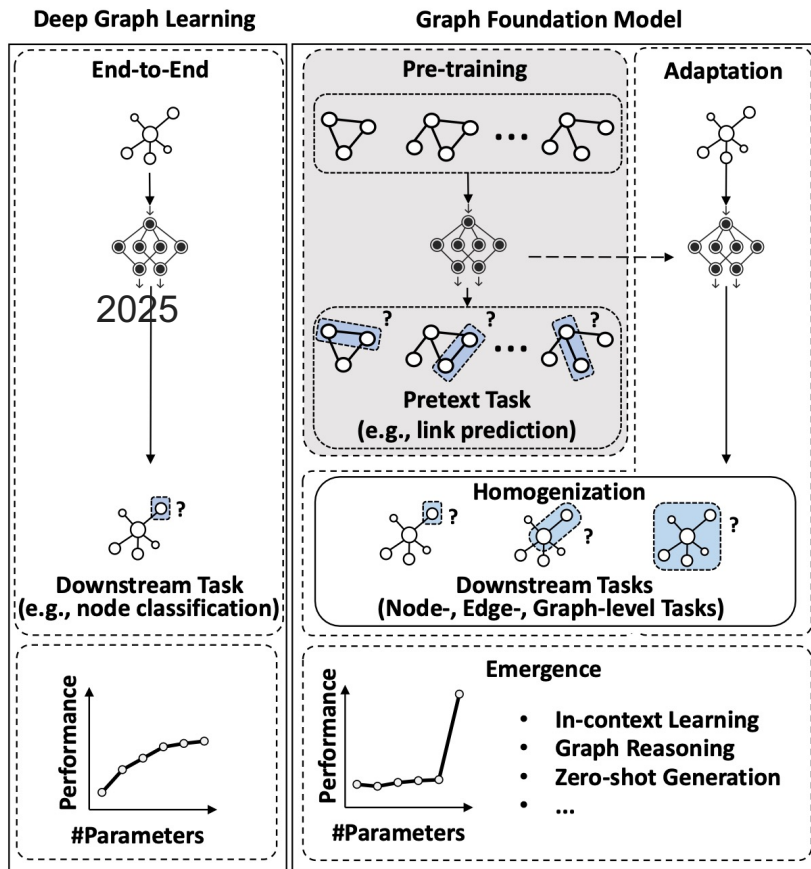


Other advanced topics: Hierarchical pooling

Literature on hierarchical pooling

- **DIFFPOOL:** [Hierarchical Graph Representation Learning with Differentiable Pooling](#), NeurIPS 2018
- [Hyperbolic Graph Convolutional Neural Networks](#), NeurIPS 2019
- [Graph U-Nets](#), ICML 2019
- [Towards Sparse Hierarchical Graph Classifiers](#), NeurIPS-W 2018
- [Deep Graph Infomax](#), ICLR 2019
- [GraphZoom: A Multi-level Spectral Approach for Accurate and Scalable Graph Embedding](#), ICLR 2020
- [StructPool: Structured Graph Pooling via Conditional Random Fields](#), ICLR 2020
- [Memory-Based Graph Networks](#), ICLR 2020

Other advanced topics: Graph Foundation Models



Liu, Jiawei, et al. "Graph Foundation Models: Concepts, Opportunities and Challenges." *IEEE TPAMI* 2025

Other advanced topics

- Physical simulations (course 13)
- Knowledge graph reasoning
- GNN for algorithms
- GNN for large-scale graphs
- Graph generations (course 6)
- Few-shot learning
- Causal Learning
- Multi-agents (course 7)

Useful materials

- PyG: <https://pytorch-geometric.readthedocs.io/en/latest/index.html>
- Course: [Stanford CS224W, Machine Learning with Graphs](#)

Summary

1. Introduction
2. Graph representation
3. Tasks
 - a. Node-level: classification and regression
 - b. Link prediction
 - c. Graph-level
4. Graph Neural Networks: Fundamentals
 - a. Overview
 - b. Aggregation
 - c. GCN
 - d. GraphSAGE
 - e. GAT

Summary

5. Useful Techniques

- a. Dropout
- b. Skip connections
- c. Virtual node
- d. Neighborhood sampling

6. Advanced topics

- a. Neural subgraph matching
- b. Graph Transformers